

Algoritmos y Estructuras de Datos. ¶

- Clase 6 - Algoritmos de Búsqueda y Ordenamiento -

Ejercicios

Ejercicio 1: Búsqueda Ternaria

link: https://es.wikipedia.org/wiki/B%C3%BAsqueda_ternaria
(https://es.wikipedia.org/wiki/B%C3%BAsqueda_ternaria)

La **búsqueda ternaria** determina que el extremo que se busca, no puede estar en el primer tercio del dominio o que no puede estar en el último tercio del dominio, luego se repite el proceso en los dos tercios restantes. Es decir, dividiremos la lista en 3 '*sub-listas*'.

- a) Implementar el Algoritmo de Búsqueda Ternaria.

Ejercicio 2: Buble Sort

link: <https://sites.google.com/site/programacioniuno/temario/unidad-6---anlisis-de-algoritmos/algoritmo-bubblesort>
(<https://sites.google.com/site/programacioniuno/temario/unidad-6---anlisis-de-algoritmos/algoritmo-bubblesort>) El algoritmo de ordenamiento por burbujeo (o bubblesort) es un algoritmo "simple" de ordenamiento. La idea es que se van a comparar pares consecutivos de elementos, y en caso en que estén desordenados, es decir que el primer elemento (izquierda) sea mayor al segundo (derecha), se hará un intercambio de posiciones.

Entonces, si tenemos que comparar cada elemento con el siguiente, podríamos pensar que el algoritmo es:

Iterar cada elemento de la lista (hasta el anteúltimo), e ir comparándolo con el siguiente. Cambiarlos de posiciones si están desordenados

Entonces, no nos alcanza con una única iteración. Así que hacemos otra implementación del algoritmo que haga lo mismo que la anterior, solo que varias veces. Cuántas ? N, donde N es la cantidad de elementos que tiene la lista.

Algoritmo de ordenamiento. Burbuja(Bubble Sort en inglés) es un sencillo algoritmo de ordenamiento. Funciona revisando cada elemento de la lista que va a ser ordenada con el siguiente, intercambiándolos de posición si están en el orden equivocado. Es necesario revisar varias veces toda la lista hasta que

6 5 3 1 8 7 2 4

no se necesiten más intercambios, lo cual significa que la lista está ordenada.

Este algoritmo obtiene su nombre de la forma con la que suben por la lista los elementos durante los intercambios, como si fueran pequeñas burbujas.

También es conocido como el método del intercambio directo. Dado que solo usa comparaciones para operar elementos, se lo considera un algoritmo de comparación, siendo el más sencillo de implementar.

- a) Implementar el Algoritmo de Ordenamiento Burbuja.

6 5 3 1 8 7 2 4

Ejercicio 3: Ordenamiento Shell (Shell Sort)

link: https://es.wikipedia.org/wiki/Ordenamiento_Shell (https://es.wikipedia.org/wiki/Ordenamiento_Shell)

El Shell sort es una generalización del ordenamiento por inserción, teniendo en cuenta dos observaciones:

- 1) El ordenamiento por inserción es eficiente si la entrada está "casi ordenada".
- 2) El ordenamiento por inserción es ineficiente, en general, porque mueve los valores solo una posición cada vez.

El algoritmo Shell sort mejora el ordenamiento por inserción comparando elementos separados por un espacio de varias posiciones. Esto permite que un elemento haga "pasos más grandes" hacia su posición esperada. Los pasos múltiples sobre los datos se hacen con tamaños de espacio cada vez más pequeños. El último paso del Shell sort es un simple ordenamiento por inserción, pero para entonces, ya está garantizado que los datos del vector están casi ordenados.

- a) Implementar el Algoritmo de Ordenamiento Shell.

Ejercicio 4: Quick Sort Mejorado,

Implementar el algoritmo de ordenamiento Quick Sort, utilizando el método de '*mediana de tres*' para la selección del pivote.